

Week 9: Sales and Customer Analytics System

The e-commerce company wants to analyze its sales data to understand customer purchasing behavior, product performance, and revenue trends. The management team needs reports to identify top customers, best-selling products, and profitable categories.

1. Apply SQL Aggregate Functions

Aggregate functions perform calculations on multiple records and return a single summarized value.

Functions Covered: Function	Purpose	Example
COUNT()	Counts number of records	Total customers/orders
SUM()	Calculates total value	Total sales amount
AVG()	Finds average value	Average order value
MIN()	Finds minimum value	Lowest product price
MAX()	Finds maximum value	Highest produ

2. Generate Total Sales Reports

Tasks:

Write SQL queries to calculate:

1. Total number of orders placed.

```
SQL> SELECT COUNT(*) AS Number_Of_Orders_Placed FROM Orders;
NUMBER_OF_ORDERS_PLACED
-----
14
```

2. Total revenue generated from sales.

```
SQL> SELECT SUM(TotalAmount) AS Total_Revenue FROM Orders;
TOTAL_REVENUE
-----
206200
```

3. Average order value.

```
SQL> SELECT ROUND(AVG(TotalAmount),2) AS Average_Order_Value FROM Orders;
```

AVERAGE_ORDER_VALUE
14728.57

4. Highest and lowest order amount.

```
SQL> SELECT MAX(TotalAmount) AS Highest_Order,
2 MIN(TotalAmount) AS Lowest_Order
3 FROM Orders;
```

HIGHEST_ORDER	LOWEST_ORDER
55850	2450

5. Total sales for a specific period.

```
SQL> SELECT SUM(TotalAmount) AS Total_Revenue FROM Orders
2 WHERE OrderDate BETWEEN DATE '2026-06-01' AND DATE '2026-08-31';
```

TOTAL_REVENUE
37450

3. Customer Purchase Analysis

1. Total number of orders placed by each customer.

```
SQL> SELECT
2 C.CUSTOMER_ID,
3 C.FIRST_NAME || ' ' || C.LAST_NAME AS CUSTOMER_NAME,
4 COUNT(O.ORDERID) AS TOTAL_ORDERS
5 FROM CUSTOMER C
6 LEFT JOIN ORDERS O
7 ON C.CUSTOMER_ID = O.CUSTOMERID
8 GROUP BY
9 C.CUSTOMER_ID,
10 C.FIRST_NAME,
11 C.LAST_NAME
12 ORDER BY TOTAL_ORDERS DESC;
```

CUSTOMER_I	CUSTOMER_NAME	TOTAL_ORDERS
CUST000001	Arjun Kumar	1
CUST000015	Nithin Murugan	1
CUST000003	Rahul Menon	1
CUST000004	Ananya Iyer	1
CUST000005	Vikram Reddy	1
CUST000006	Meera Krishnan	1
CUST000007	Suresh Nair	1
CUST000008	Kavya Rajan	1
CUST000009	Karthik Subramanian	1
CUST000010	Divya Pillai	1
CUST000011	Aditya Verma	1
CUST000012	Sneha Patel	1
CUST000002	Priya Sharma	1
CUST000014	Lakshmi Venkatesan	1
CUST000013	Rohan Bose	0

15 rows selected.

2. Total amount spent by each customer.

```
SQL> SELECT
2     C.CUSTOMER_ID,
3     C.FIRST_NAME || ' ' || C.LAST_NAME AS CUSTOMER_NAME,
4     NVL(SUM(O.TOTALAMOUNT), 0) AS TOTAL_AMOUNT_SPENT
5 FROM CUSTOMER C
6 LEFT JOIN ORDERS O
7     ON C.CUSTOMER_ID = O.CUSTOMERID
8 GROUP BY
9     C.CUSTOMER_ID,
10    C.FIRST_NAME,
11    C.LAST_NAME
12 ORDER BY TOTAL_AMOUNT_SPENT DESC;
```

CUSTOMER_I	CUSTOMER_NAME	TOTAL_AMOUNT_SPENT
CUST000001	Arjun Kumar	55850
CUST000014	Lakshmi Venkatesan	28000
CUST000003	Rahul Menon	28000
CUST000005	Vikram Reddy	22000
CUST000009	Karthik Subramanian	18500
CUST000015	Nithin Murugan	8350
CUST000004	Ananya Iyer	8300
CUST000011	Aditya Verma	7000
CUST000006	Meera Krishnan	7000
CUST000007	Suresh Nair	6500
CUST000008	Kavya Rajan	5700
CUSTOMER_I	CUSTOMER_NAME	TOTAL_AMOUNT_SPENT
CUST000010	Divya Pillai	5200
CUST000002	Priya Sharma	3350
CUST000012	Sneha Patel	2450
CUST000013	Rohan Bose	0

15 rows selected.

NVL() function replaces NULL with a specified value. Here for the customer with no purchase the Amount Spent will be zero.

3. Average spending per customer.

```

SQL> SELECT
  2     C.CUSTOMER_ID,
  3     C.FIRST_NAME || ' ' || C.LAST_NAME AS CUSTOMER_NAME,
  4     ROUND(NVL(AVG(O.TOTALAMOUNT), 0), 2) AS AVERAGE_SPENDING
  5 FROM CUSTOMER C
  6 LEFT JOIN ORDERS O
  7     ON C.CUSTOMER_ID = O.CUSTOMERID
  8 GROUP BY
  9     C.CUSTOMER_ID,
 10     C.FIRST_NAME,
 11     C.LAST_NAME
 12 ORDER BY AVERAGE_SPENDING DESC;

```

CUSTOMER_I	CUSTOMER_NAME	AVERAGE_SPENDING
CUST000001	Arjun Kumar	55850
CUST000014	Lakshmi Venkatesan	28000
CUST000003	Rahul Menon	28000
CUST000005	Vikram Reddy	22000
CUST000009	Karthik Subramanian	18500
CUST000015	Nithin Murugan	8350
CUST000004	Ananya Iyer	8300
CUST000011	Aditya Verma	7000
CUST000006	Meera Krishnan	7000
CUST000007	Suresh Nair	6500
CUST000008	Kavya Rajan	5700
CUST000010	Divya Pillai	5200
CUST000002	Priya Sharma	3350
CUST000012	Sneha Patel	2450
CUST000013	Rohan Bose	0

15 rows selected.

4. Customers with maximum purchase amount.

```

SQL> SELECT
  2     C.CUSTOMER_ID,
  3     C.FIRST_NAME || ' ' || C.LAST_NAME AS CUSTOMER_NAME,
  4     SUM(O.TOTALAMOUNT) AS TOTAL_PURCHASE_AMOUNT
  5 FROM CUSTOMER C
  6 JOIN ORDERS O
  7     ON C.CUSTOMER_ID = O.CUSTOMERID
  8 GROUP BY
  9     C.CUSTOMER_ID,
 10     C.FIRST_NAME,
 11     C.LAST_NAME
 12 HAVING SUM(O.TOTALAMOUNT) = (
 13     SELECT MAX(TOTAL_SPENDING)
 14     FROM (
 15         SELECT SUM(TOTALAMOUNT) AS TOTAL_SPENDING
 16         FROM ORDERS
 17         GROUP BY CUSTOMERID
 18     )
 19 );

```

CUSTOMER_I	CUSTOMER_NAME	TOTAL_PURCHASE_AMOUNT
CUST000001	Arjun Kumar	55850

5. Customers with fewer purchases.

```

SQL> SELECT
  2     C.CUSTOMER_ID,
  3     C.FIRST_NAME || ' ' || C.LAST_NAME AS CUSTOMER_NAME,
  4     COUNT(O.ORDERID) AS NUMBER_OF_ORDERS
  5 FROM CUSTOMER C
  6 LEFT JOIN ORDERS O
  7     ON C.CUSTOMER_ID = O.CUSTOMERID
  8 GROUP BY
  9     C.CUSTOMER_ID,
 10     C.FIRST_NAME,
 11     C.LAST_NAME
 12 HAVING COUNT(O.ORDERID) <= 1
 13 ORDER BY NUMBER_OF_ORDERS;

```

CUSTOMER_I	CUSTOMER_NAME	NUMBER_OF_ORDERS
CUST000013	Rohan Bose	0
CUST000015	Nithin Murugan	1
CUST000003	Rahul Menon	1
CUST000004	Ananya Iyer	1
CUST000005	Vikram Reddy	1
CUST000006	Meera Krishnan	1
CUST000007	Suresh Nair	1
CUST000008	Kavya Rajan	1
CUST000009	Karthik Subramanian	1
CUST000010	Divya Pillai	1
CUST000011	Aditya Verma	1
CUST000012	Sneha Patel	1
CUST000014	Lakshmi Venkatesan	1
CUST000001	Arjun Kumar	1
CUST000002	Priya Sharma	1

15 rows selected.

4. Find Top Customers Based on Purchase Amount

1. Top 5 customers based on total spending.

```

SQL> SELECT
  2     C.CUSTOMER_ID,
  3     C.FIRST_NAME || ' ' || C.LAST_NAME AS CUSTOMER_NAME,
  4     SUM(O.TOTALAMOUNT) AS TOTAL_SPENDING
  5 FROM CUSTOMER C
  6 JOIN ORDERS O
  7     ON C.CUSTOMER_ID = O.CUSTOMERID
  8 GROUP BY
  9     C.CUSTOMER_ID,
 10     C.FIRST_NAME,
 11     C.LAST_NAME
 12 ORDER BY TOTAL_SPENDING DESC
 13 FETCH FIRST 5 ROWS ONLY;

```

CUSTOMER_I	CUSTOMER_NAME	TOTAL_SPENDING
CUST000001	Arjun Kumar	55850
CUST000003	Rahul Menon	28000
CUST000014	Lakshmi Venkatesan	28000
CUST000005	Vikram Reddy	22000
CUST000009	Karthik Subramanian	18500

2. Customers with maximum number of orders.

```
SQL> SELECT
  2     C.CUSTOMER_ID,
  3     C.FIRST_NAME || ' ' || C.LAST_NAME AS CUSTOMER_NAME,
  4     COUNT(O.ORDERID) AS NUMBER_OF_ORDERS
  5 FROM CUSTOMER C
  6 JOIN ORDERS O
  7     ON C.CUSTOMER_ID = O.CUSTOMERID
  8 GROUP BY
  9     C.CUSTOMER_ID,
 10     C.FIRST_NAME,
 11     C.LAST_NAME
 12 HAVING COUNT(O.ORDERID) = (
 13     SELECT MAX(ORDER_COUNT)
 14     FROM (
 15         SELECT COUNT(*) AS ORDER_COUNT
 16         FROM ORDERS
 17         GROUP BY CUSTOMERID
 18     )
 19 )
 20 ORDER BY CUSTOMER_NAME;
```

CUSTOMER_I	CUSTOMER_NAME	NUMBER_OF_ORDERS
CUST000011	Aditya Verma	1
CUST000004	Ananya Iyer	1
CUST000001	Arjun Kumar	1
CUST000010	Divya Pillai	1
CUST000009	Karthik Subramanian	1
CUST000008	Kavya Rajan	1
CUST000014	Lakshmi Venkatesan	1
CUST000006	Meera Krishnan	1
CUST000015	Nithin Murugan	1
CUST000002	Priya Sharma	1
CUST000003	Rahul Menon	1

CUSTOMER_I	CUSTOMER_NAME	NUMBER_OF_ORDERS
CUST000012	Sneha Patel	1
CUST000007	Suresh Nair	1
CUST000005	Vikram Reddy	1

14 rows selected.

3. Frequent customers.

```

SQL> SELECT
  2     C.CUSTOMER_ID,
  3     C.FIRST_NAME || ' ' || C.LAST_NAME AS CUSTOMER_NAME,
  4     COUNT(O.ORDERID) AS NUMBER_OF_ORDERS
  5 FROM CUSTOMER C
  6 JOIN ORDERS O
  7     ON C.CUSTOMER_ID = O.CUSTOMERID
  8 GROUP BY
  9     C.CUSTOMER_ID,
 10     C.FIRST_NAME,
 11     C.LAST_NAME
 12 HAVING COUNT(O.ORDERID) >= 2
 13 ORDER BY NUMBER_OF_ORDERS DESC;

no rows selected

```

4. High-value customers.

```

SQL> SELECT
  2     C.CUSTOMER_ID,
  3     C.FIRST_NAME || ' ' || C.LAST_NAME AS CUSTOMER_NAME,
  4     COUNT(O.ORDERID) AS NUMBER_OF_ORDERS,
  5     SUM(O.TOTALAMOUNT) AS TOTAL_SPENDING
  6 FROM CUSTOMER C
  7 JOIN ORDERS O
  8     ON C.CUSTOMER_ID = O.CUSTOMERID
  9 GROUP BY
 10     C.CUSTOMER_ID,
 11     C.FIRST_NAME,
 12     C.LAST_NAME
 13 HAVING SUM(O.TOTALAMOUNT) > 20000
 14 ORDER BY TOTAL_SPENDING DESC;

```

CUSTOMER_I	CUSTOMER_NAME	NUMBER_OF_ORDERS

TOTAL_SPENDING		

CUST000001	Arjun Kumar	1
	55850	
CUST000014	Lakshmi Venkatesan	1
	28000	
CUST000003	Rahul Menon	1
	28000	

CUSTOMER_I	CUSTOMER_NAME	NUMBER_OF_ORDERS

TOTAL_SPENDING		

CUST000005	Vikram Reddy	1
	22000	

5. Identify Best-Selling Products

1. Products with maximum sales quantity.

```
SQL> SELECT
  2     P.PRODUCT_ID,
  3     P.PRODUCT_NAME,
  4     SUM(OD.QUANTITY) AS TOTAL_QUANTITY_SOLD
  5 FROM PRODUCT P
  6 JOIN ORDERDETAIL OD
  7     ON P.PRODUCT_ID = OD.PRODUCTID
  8 GROUP BY
  9     P.PRODUCT_ID,
 10     P.PRODUCT_NAME
 11 ORDER BY TOTAL_QUANTITY_SOLD DESC;
```

PRODUCT	PRODUCT_NAME	TOTAL_QUANTITY_SOLD
P000006	USB Cable	6
P000005	Smartphone	6
P000015	Wireless Earbuds	3
P000008	Smart Watch	2
P000012	External Hard Disk	2
P000009	Headphones	2
P000001	Laptop	1
P000013	Gaming Monitor	1
P000010	Tablet	1
P000014	Laptop Stand	1
P000002	Wireless Mouse	1
P000003	Mechanical Keyboard	1
P000004	Bluetooth Speaker	1

13 rows selected.

2. Products generating highest revenue.

```
SQL> SELECT
  2     P.PRODUCT_ID,
  3     P.PRODUCT_NAME,
  4     SUM(OD.QUANTITY) AS QUANTITY_SOLD,
  5     SUM(OD.QUANTITY * OD.PRICE) AS TOTAL_REVENUE
  6 FROM PRODUCT P
  7 JOIN ORDERDETAIL OD
  8     ON P.PRODUCT_ID = OD.PRODUCTID
  9 GROUP BY
 10     P.PRODUCT_ID,
 11     P.PRODUCT_NAME
 12 ORDER BY TOTAL_REVENUE DESC;
```

PRODUCT	PRODUCT_NAME	QUANTITY_SOLD
P000005	Smartphone	6
P000001	Laptop	1
P000010	Tablet	1

PRODUCT	PRODUCT_NAME	QUANTITY_SOLD
TOTAL_REVENUE		
P000013	Gaming Monitor 18500	1
P000008	Smart Watch 13000	2
P000015	Wireless Earbuds 10500	3
TOTAL_REVENUE		
P000012	External Hard Disk 10400	2
P000009	Headphones 8400	2
P000004	Bluetooth Speaker 3200	1
TOTAL_REVENUE		
P000006	USB Cable 2700	6
P000003	Mechanical Keyboard 2500	1
P000014	Laptop Stand 1500	1
TOTAL_REVENUE		
P000002	Wireless Mouse 850	1

13 rows selected.

3. Least-selling products.

```
SQL> SELECT
  2     P.PRODUCT_ID,
  3     P.PRODUCT_NAME,
  4     NVL(SUM(OD.QUANTITY), 0) AS QUANTITY_SOLD
  5 FROM PRODUCT P
  6 LEFT JOIN ORDERDETAIL OD
  7     ON P.PRODUCT_ID = OD.PRODUCTID
  8 GROUP BY
  9     P.PRODUCT_ID,
10     P.PRODUCT_NAME
11 ORDER BY QUANTITY_SOLD ASC;
```

PRODUCT	PRODUCT_NAME	QUANTITY_SOLD
P000007	Power Bank	0
P000011	Web Camera	0
P000004	Bluetooth Speaker	1
P000003	Mechanical Keyboard	1
P000001	Laptop	1
P000002	Wireless Mouse	1
P000014	Laptop Stand	1
P000013	Gaming Monitor	1
P000010	Tablet	1
P000012	External Hard Disk	2
P000009	Headphones	2

PRODUCT	PRODUCT_NAME	QUANTITY_SOLD
P000008	Smart Watch	2
P000015	Wireless Earbuds	3
P000005	Smartphone	6
P000006	USB Cable	6

15 rows selected

4. Products requiring promotion.

```
SQL> SELECT
  2     P.PRODUCT_ID,
  3     P.PRODUCT_NAME,
  4     P.PRICE,
  5     P.STOCK_QUANTITY,
  6     NVL(SUM(OD.QUANTITY), 0) AS QUANTITY_SOLD
  7 FROM PRODUCT P
  8 LEFT JOIN ORDERDETAIL OD
  9     ON P.PRODUCT_ID = OD.PRODUCTID
 10 GROUP BY
 11     P.PRODUCT_ID,
 12     P.PRODUCT_NAME,
 13     P.PRICE,
 14     P.STOCK_QUANTITY
 15 HAVING NVL(SUM(OD.QUANTITY), 0) <= 1
 16        AND P.STOCK_QUANTITY > 0
 17 ORDER BY QUANTITY_SOLD, P.PRODUCT_NAME;
```

PRODUCT	PRODUCT_NAME	PRICE
P000007	Power Bank	1800
75	0	

P000011	Web Camera	2800
50	0	

P000004	Bluetooth Speaker	3200
45	1	

PRODUCT	PRODUCT_NAME	PRICE
P000013	Gaming Monitor	18500
20	1	

P000001	Laptop	55000
25	1	

P000014	Laptop Stand	1500
80	1	

PRODUCT	PRODUCT_NAME	PRICE
P000003	Mechanical Keyboard	2500
60	1	

P000010	Tablet	22000
30	1	

P000002	Wireless Mouse	850
100	1	

9 rows selected.

6. Perform Category-wise Sales Analysis

1. Total sales generated by each category.

```
SQL> SELECT
  2     C.CATEGORY_NAME,
  3     SUM(OD.QUANTITY * OD.PRICE) AS TOTAL_SALES
  4 FROM CATEGORY C
  5 JOIN PRODUCT P
  6     ON C.CATEGORY_ID = P.CATEGORY_ID
  7 JOIN ORDERDETAIL OD
  8     ON P.PRODUCT_ID = OD.PRODUCTID
  9 GROUP BY
 10     C.CATEGORY_ID,
 11     C.CATEGORY_NAME
 12 ORDER BY TOTAL_SALES DESC;
```

CATEGORY_NAME	TOTAL_SALES
Mobile Devices	190000
Laptops	55000
Audio Devices	22100
Monitors	18500
Wearables	13000
Storage Devices	10400
Computer Accessories	3350
Power Accessories	2700
Laptop Accessories	1500

9 rows selected.

2. Number of products sold in each category.

```
SQL> SELECT
  2     C.CATEGORY_NAME,
  3     SUM(OD.QUANTITY) AS PRODUCTS_SOLD
  4 FROM CATEGORY C
  5 JOIN PRODUCT P
  6     ON C.CATEGORY_ID = P.CATEGORY_ID
  7 JOIN ORDERDETAIL OD
  8     ON P.PRODUCT_ID = OD.PRODUCTID
  9 GROUP BY
 10     C.CATEGORY_ID,
 11     C.CATEGORY_NAME
 12 ORDER BY PRODUCTS_SOLD DESC;
```

CATEGORY_NAME	PRODUCTS_SOLD
Mobile Devices	7
Power Accessories	6
Audio Devices	6
Computer Accessories	2
Wearables	2
Storage Devices	2
Laptop Accessories	1
Laptops	1
Monitors	1

9 rows selected.

3. Highest revenue-generating category.

```
SQL> SELECT
  2     C.CATEGORY_NAME,
  3     SUM(OD.QUANTITY * OD.PRICE) AS TOTAL_REVENUE
  4 FROM CATEGORY C
  5 JOIN PRODUCT P
  6     ON C.CATEGORY_ID = P.CATEGORY_ID
  7 JOIN ORDERDETAIL OD
  8     ON P.PRODUCT_ID = OD.PRODUCTID
  9 GROUP BY
 10     C.CATEGORY_ID,
 11     C.CATEGORY_NAME
 12 HAVING SUM(OD.QUANTITY * OD.PRICE) = (
 13     SELECT MAX(CATEGORY_REVENUE)
 14     FROM (
 15         SELECT SUM(OD2.QUANTITY * OD2.PRICE) AS CATEGORY_REVENUE
 16         FROM CATEGORY C2
 17         JOIN PRODUCT P2
 18             ON C2.CATEGORY_ID = P2.CATEGORY_ID
 19         JOIN ORDERDETAIL OD2
 20             ON P2.PRODUCT_ID = OD2.PRODUCTID
 21         GROUP BY C2.CATEGORY_ID
 22     )
 23 );
```

CATEGORY_NAME	TOTAL_REVENUE
Mobile Devices	190000

4. Average sales per category.

```
SQL> SELECT
  2     C.CATEGORY_NAME,
  3     ROUND(AVG(OD.QUANTITY * OD.PRICE), 2) AS AVERAGE_SALES
  4 FROM CATEGORY C
  5 JOIN PRODUCT P
  6     ON C.CATEGORY_ID = P.CATEGORY_ID
  7 JOIN ORDERDETAIL OD
  8     ON P.PRODUCT_ID = OD.PRODUCTID
  9 GROUP BY
 10     C.CATEGORY_ID,
 11     C.CATEGORY_NAME
 12 ORDER BY AVERAGE_SALES DESC;
```

CATEGORY_NAME	AVERAGE_SALES
Mobile Devices	95000
Laptops	55000
Monitors	18500
Wearables	6500
Storage Devices	5200
Audio Devices	3683.33
Computer Accessories	1675
Laptop Accessories	1500
Power Accessories	900

9 rows selected.

7. Generate Business Analytics Reports

Report 1: Sales Performance Report

- Total sales
- Number of orders
- Average order value

Highest order value

```
SQL> SELECT
  2     SUM(TOTALAMOUNT) AS TOTAL_SALES,
  3     COUNT(ORDERID) AS NUMBER_OF_ORDERS,
  4     ROUND(AVG(TOTALAMOUNT), 2) AS AVERAGE_ORDER_VALUE,
  5     MAX(TOTALAMOUNT) AS HIGHEST_ORDER_VALUE
  6 FROM ORDERS;
```

TOTAL_SALES	NUMBER_OF_ORDERS	AVERAGE_ORDER_VALUE	HIGHEST_ORDER_VALUE
206200	14	14728.57	55850

Report 2: Customer Analytics Report

- Customer name
- Number of orders
- Total spending
- Purchase frequency

```
SQL> SELECT
  2     C.FIRST_NAME || ' ' || C.LAST_NAME AS CUSTOMER_NAME,
  3     COUNT(O.ORDERID) AS NUMBER_OF_ORDERS,
  4     NVL(SUM(O.TOTALAMOUNT), 0) AS TOTAL_SPENDING,
  5     COUNT(O.ORDERID) AS PURCHASE_FREQUENCY
  6 FROM CUSTOMER C
  7 LEFT JOIN ORDERS O
  8     ON C.CUSTOMER_ID = O.CUSTOMERID
  9 GROUP BY
 10     C.CUSTOMER_ID,
 11     C.FIRST_NAME,
 12     C.LAST_NAME
 13 ORDER BY TOTAL_SPENDING DESC;
```

CUSTOMER_NAME	NUMBER_OF_ORDERS	TOTAL_SPENDING
---------------	------------------	----------------

PURCHASE_FREQUENCY

Arjun Kumar	1	55850
-------------	---	-------

Lakshmi Venkatesan	1	28000
--------------------	---	-------

Rahul Menon	1	28000
-------------	---	-------

CUSTOMER_NAME	NUMBER_OF_ORDERS	TOTAL_SPENDING
---------------	------------------	----------------

PURCHASE_FREQUENCY

Vikram Reddy	1	22000
--------------	---	-------

Karthik Subramanian	1	18500
---------------------	---	-------

Nithin Murugan	1	8350
----------------	---	------

CUSTOMER_NAME	NUMBER_OF_ORDERS	TOTAL_SPENDING
PURCHASE_FREQUENCY		
Ananya Iyer	1	8300
Aditya Verma	1	7000
Meera Krishnan	1	7000

CUSTOMER_NAME	NUMBER_OF_ORDERS	TOTAL_SPENDING
PURCHASE_FREQUENCY		
Suresh Nair	1	6500
Kavya Rajan	1	5700
Divya Pillai	1	5200

CUSTOMER_NAME	NUMBER_OF_ORDERS	TOTAL_SPENDING
PURCHASE_FREQUENCY		
Priya Sharma	1	3350
Sneha Patel	1	2450
Rohan Bose	0	0

15 rows selected.

Report 3: Product Performance Report

- Product name
- Quantity sold
- Revenue generated

```
SQL> SELECT
2     P.PRODUCT_NAME,
3     SUM(OD.QUANTITY) AS QUANTITY_SOLD,
4     SUM(OD.QUANTITY * OD.PRICE) AS REVENUE_GENERATED
5 FROM PRODUCT P
6 JOIN ORDERDETAIL OD
7     ON P.PRODUCT_ID = OD.PRODUCTID
8 GROUP BY
9     P.PRODUCT_ID,
10    P.PRODUCT_NAME
11 ORDER BY REVENUE_GENERATED DESC;
```

PRODUCT_NAME	QUANTITY_SOLD
REVENUE_GENERATED	
Smartphone	6
168000	
Laptop	1
55000	
Tablet	1
22000	
PRODUCT_NAME	QUANTITY_SOLD
REVENUE_GENERATED	
Gaming Monitor	1
18500	
Smart Watch	2
13000	
Wireless Earbuds	3
10500	
PRODUCT_NAME	QUANTITY_SOLD
REVENUE_GENERATED	
External Hard Disk	2
10400	
Headphones	2
8400	
Bluetooth Speaker	1
3200	
PRODUCT_NAME	QUANTITY_SOLD
REVENUE_GENERATED	
USB Cable	6
2700	
Mechanical Keyboard	1
2500	
Laptop Stand	1
1500	

PRODUCT_NAME	QUANTITY_SOLD
REVENUE_GENERATED	
Wireless Mouse	1
850	

13 rows selected.

Report 4: Category Analysis Report

- Category name
- Total products sold
- Total revenue

```
SQL> SELECT
  2     C.CATEGORY_NAME,
  3     SUM(OD.QUANTITY) AS TOTAL_PRODUCTS_SOLD,
  4     SUM(OD.QUANTITY * OD.PRICE) AS TOTAL_REVENUE
  5 FROM CATEGORY C
  6 JOIN PRODUCT P
  7   ON C.CATEGORY_ID = P.CATEGORY_ID
  8 JOIN ORDERDETAIL OD
  9   ON P.PRODUCT_ID = OD.PRODUCTID
 10 GROUP BY
 11     C.CATEGORY_ID,
 12     C.CATEGORY_NAME
 13 ORDER BY TOTAL_REVENUE DESC;
```

CATEGORY_NAME	TOTAL_PRODUCTS_SOLD	TOTAL_REVENUE
Mobile Devices	7	190000
Laptops	1	55000
Audio Devices	6	22100
Monitors	1	18500
Wearables	2	13000
Storage Devices	2	10400
Computer Accessories	2	3350
Power Accessories	6	2700
Laptop Accessories	1	1500

9 rows selected.